The Complete Handbook

Open Knowledge Format, RAG, and Agentic Retrieval — A Practitioner's Guide

google-okf-tutorial

2026-07-05

Contents

About This Handbook

This handbook consolidates everything built and learned in the `google-okf-tutorial` repository into a single, self-contained reference: definitions, theory (cited to primary sources), full code, real measured results, and the specific bugs that were found and fixed along the way. It accompanies two fully-executed Jupyter notebooks and a standalone comparison report:

- `google_okf_zero_to_mastery.ipynb` — Google's Open Knowledge Format (OKF) v0.1, implemented from scratch, solving a real problem: turning an undocumented production dataset into an agent-consumable knowledge bundle.
- `agentic_rag_chromadb.ipynb` — the traditional alternative: no knowledge-structuring layer, flat text chunks in ChromaDB, consumed by both simple and agentic (tool-calling) retrieval.
- `OKF_vs_RAG_Comparison.md/.pdf` — a focused, standalone comparison of the two approaches.

Every number in this handbook is copied from an executed cell output in one of the two notebooks, not estimated or reconstructed from memory.

How to use this handbook: Part I defines every term used elsewhere in this document. Part II explains the theory, with citations. Parts III and IV walk through each notebook's architecture and reasoning, with representative code inline and complete code in the Appendices. Part V compares the two approaches. Part VI is a catalog of real bugs found during development, with root causes — this is arguably the most practically useful section for anyone building similar systems. Part VII is a consolidated bibliography.

Part I — Definitions

OKF (Open Knowledge Format) — An open, v0.1-draft specification from Google (`GoogleCloudPlatform/knowledge-catalog`) representing knowledge as a directory of Markdown files with YAML frontmatter. Minimal by design: one required field (`type`), no runtime, no SDK.

Knowledge Bundle — A self-contained, hierarchical collection of OKF knowledge documents; the unit of distribution in OKF.

Concept — A single unit of knowledge in an OKF bundle, represented as one Markdown file. Its **Concept ID** is the file's path relative to the bundle root with the `.md` suffix removed (e.g. `tables/orders.md` → `tables/orders`).

Frontmatter — The YAML metadata block delimited by `---` at the top of an OKF concept file.

Conformance (OKF) — A bundle is conformant with OKF v0.1 iff every non-reserved `.md` file has parseable YAML frontmatter with a non-empty `type` field. All other properties (missing optional fields, unknown types, broken links) are soft guidance a conformant consumer must tolerate.

RAG (Retrieval-Augmented Generation) — An inference-time technique where a language model's output is grounded by first retrieving relevant content from an external corpus and including it in the prompt, rather than relying solely on the model's parametric memory.

Dense retrieval — Retrieval by nearest-neighbor search over vector embeddings (this project uses `nomic-embed-text` embeddings with FAISS or ChromaDB).

Sparse retrieval / BM25 — Lexical, term-frequency-based retrieval (Robertson & Spärck Jones); implemented here via the `rank_bm25` library. Catches exact-term matches dense embeddings can miss.

Hybrid retrieval — Combining dense and sparse retrieval and fusing their ranked result lists.

Reciprocal Rank Fusion (RRF) — A rank-fusion formula, $\mathrm{RRF}(d) = \sum_i \frac{1}{k+\mathrm{rank}_i(d)}$, combining multiple ranked lists into one (Cormack, Clarke & Büttcher, 2009). The constant k (60 in the original paper) is a smoothing term.

RAG-Fusion — Generating multiple reformulations of a user query with an LLM, retrieving for each, and fusing the results with RRF (Rackauckas, 2024).

HyDE (Hypothetical Document Embeddings) — Having an LLM write a hypothetical answer to a query first, then embedding *that* (rather than, or in addition to, the raw query) for retrieval — because a plausible answer paragraph sits closer in embedding space to real answer paragraphs than a short question does (Gao et al., 2022).

Listwise reranking — Giving an LLM a list of retrieved candidates at once and asking it to produce a single reordering (e.g. RankGPT, Sun et al. 2023). Effective with large models; measured in this project to be unreliable with a 4B model.

Pointwise reranking — Scoring each retrieved candidate independently, in its own LLM call, with no shared context between judgments. Adopted in this project specifically because listwise reranking was measured to fail at small model scale.

GraphRAG — Using graph structure (rather than pure vector similarity) to inform retrieval (Edge et al., Microsoft Research). This project's OKF-link expansion is a lightweight instance of the same idea, using OKF's already-authored link graph instead of an LLM-extracted one.

Agentic RAG — RAG where an LLM agent, not a fixed pipeline, decides when and how to retrieve — potentially issuing multiple searches, reformulating, or stopping when it judges it

has enough information (survey: Singh et al., 2025).

ReAct — The foundational pattern of interleaving reasoning traces with actions (e.g. tool calls) in an LLM's generation, letting reasoning guide which action to take next and the action's result inform further reasoning (Yao et al., 2022).

Tool calling / function calling — An LLM API feature (used here via Ollama, verified working with `qwen3.5:4b`) where the model can emit a structured request to invoke a named function with arguments, receive the result, and continue reasoning — the mechanism this project's agentic loop is built on.

Vector database — A database specialized for storing embeddings and performing nearest-neighbor search. This project uses **FAISS** (a library, not a server — `IndexFlatIP` specifically, exact brute-force cosine similarity via inner product on normalized vectors) in notebook 1, and **ChromaDB** (an embedded document+vector store with metadata filtering) in notebook 2.

Grounding — Constraining an LLM's output to facts actually present in provided context or computed by code, rather than the model's parametric knowledge. The central discipline of both notebooks: **facts come from code (pandas), prose comes from the LLM**, and every generated claim involving a number is checked against ground truth before being accepted.

Hallucination — An LLM stating something false with no basis in the provided context — measured directly in this project (e.g., a dataset-overview prompt that invented a "2019–2023" date range for data that actually spans 2016-2018) and mitigated by grounding + verify-then-retry-then-fallback.

Mean Reciprocal Rank (MRR) — $\frac{1}{|Q|}\sum_{q}\frac{1}{\text{rank}_q}$ where rank_q is the position of the first correct result for query q (0 if never found) — a finer-grained retrieval metric than Hit@k, used in this project's retired advanced-RAG benchmark.

Hit@k — Whether the correct/expected item appears anywhere in the top-k retrieved results.

Part II — Theory

2.1 The Open Knowledge Format, in depth

2.1.1 Motivation

Organizations accumulate knowledge about their data and systems — what a table means, how it joins to others, what a metric measures — in incompatible places: proprietary catalog UIs, wikis, code comments, and the heads of whoever last touched the system. An AI agent asked a non-trivial question has no single, consistent place to look. OKF's answer is not another service — it is a **format**: a directory of Markdown files with YAML frontmatter, chosen specifically because it is:

- **Readable by humans without tooling** — any editor, `cat`, or GitHub's renderer works.
- **Parseable by agents without bespoke SDKs** — any LLM can ingest a concept file verbatim.
- **Diffable in version control** — knowledge curation becomes a normal engineering activity: pull requests, blame, review.
- **Portable and lock-in-free** — a bundle is a directory; ship it as a tarball, host it in any repo.

2.1.2 The mechanics

A **bundle** is a directory tree of Markdown files. `index.md` (progressive-disclosure directory listing) and `log.md` (dated change history) are reserved filenames with defined structure; every other `.md` file is a **concept**. A concept has a YAML frontmatter block (only `type` is required; `title`, `description`, `resource`, `tags`, `timestamp` are recommended; producers may add arbitrary extra keys) followed by a free-form Markdown body. Three body headings — `# Schema`, `# Examples`, `# Citations` — carry conventional (not required) meaning. Concepts link to each other with ordinary Markdown links; **bundle-absolute** links (`[text](/tables/customers.md)`) are recommended because they stay valid when a file moves within its own subdirectory. Consumers must tolerate broken links — OKF is explicitly designed to stay useful as a bundle is partially generated and refactored by agents over time. **Conformance** (SPEC.md §9) requires exactly two things: every concept's frontmatter parses, and `type` is non-empty. Everything else is soft guidance.

2.1.3 What notebook 1 built against this spec

An independent implementation — not a copy of Google's own reference agent — of: the `OkfDocument` model, frontmatter render/parse round-tripping, `index.md`/`log.md` generators, link extraction/resolution, and a conformance validator that implements the exact two hard rules above plus soft warnings for broken links. See Part III and Appendix A for the full code.

2.2 RAG fundamentals

Naive RAG is: embed a query, retrieve the top-k nearest corpus items by similarity, optionally reorder them, generate an answer using them as context. It has three well-documented failure modes, each with a fix used somewhere in this project:

1. **Vocabulary mismatch** — a query and the document that answers it may share no exact terms. *Fix*: hybrid dense+sparse (BM25) retrieval, so lexical matches aren't lost to embedding fuzz.
2. **Single-phrasing brittleness** — one query embedding is one point in space; if phrasing doesn't match the corpus, retrieval silently underperforms. *Fix*: RAG-Fusion

(multi-query expansion + RRF fusion) and HyDE (embed a hypothetical answer instead of/alongside the raw query).

3. **Reranking unreliability at small model scale** — see §2.3.

2.3 Reranking: listwise vs. pointwise

Listwise reranking (RankGPT, Sun et al. 2023) asks a model to look at several candidates at once and produce one coherent ordering. It works well with large models (GPT-3.5/4) but the literature notes real reliability and variance issues with smaller, undistilled models — exactly what this project measured directly: `qwen3.5:4b`, asked to freely reorder 8 candidates, dropped the single most relevant document for a question dense retrieval had already ranked correctly. **Pointwise** reranking — one independent relevance judgment per candidate, no shared context between judgments — is a strictly simpler task per call, and this project adopted it specifically because the simpler task proved more reliable at this model scale (Part VI, Bug #1).

2.4 Graph-aware retrieval

GraphRAG (Edge et al., Microsoft Research) uses graph structure — not just vector similarity — to find relevant context, typically by having an LLM extract an entity graph from unstructured text and detecting communities within it. This project's OKF-link expansion is a lightweight instance of the same underlying idea, simplified because OKF concepts already carry an *authored* link graph (every table lists its real joins; every metric lists its real source tables) — there is no graph to extract, only one to traverse.

2.5 Agentic RAG and ReAct

ReAct (Yao et al., 2022) interleaves reasoning traces with actions (e.g., tool calls): reasoning helps the model decide what to do next; actions let it pull in new information the reasoning can then build on. **Agentic RAG** (survey: Singh et al., 2025) applies this to retrieval specifically — instead of a fixed "retrieve once, generate" pipeline, the model decides for itself whether to search, how to phrase the search, and whether to search again. Notebook 2 implements this with Ollama's native tool-calling API (verified working with `qwen3.5:4b` before being relied upon), giving the model one tool, `search_knowledge_base`, in a loop bounded at 4 iterations.

2.6 Vector databases: FAISS vs. ChromaDB

FAISS (`IndexFlatIP` in this project) is a library, not a server: an in-process, exact (brute-force) nearest-neighbor index over normalized vectors, with cosine similarity computed as inner product. At 16 vectors, exact brute-force search is instant and there is no reason to reach for an approximate index. **ChromaDB** is an embedded document store with a first-class notion of a "collection" (documents + metadata + embeddings together), a pluggable embedding function interface, and metadata filtering — closer to what most real-world RAG deployments reach for, at the cost of a heavier dependency (embedded SQLite + an ONNX runtime) and a genuine architectural requirement: a running database process, however lightweight, rather than "just files."

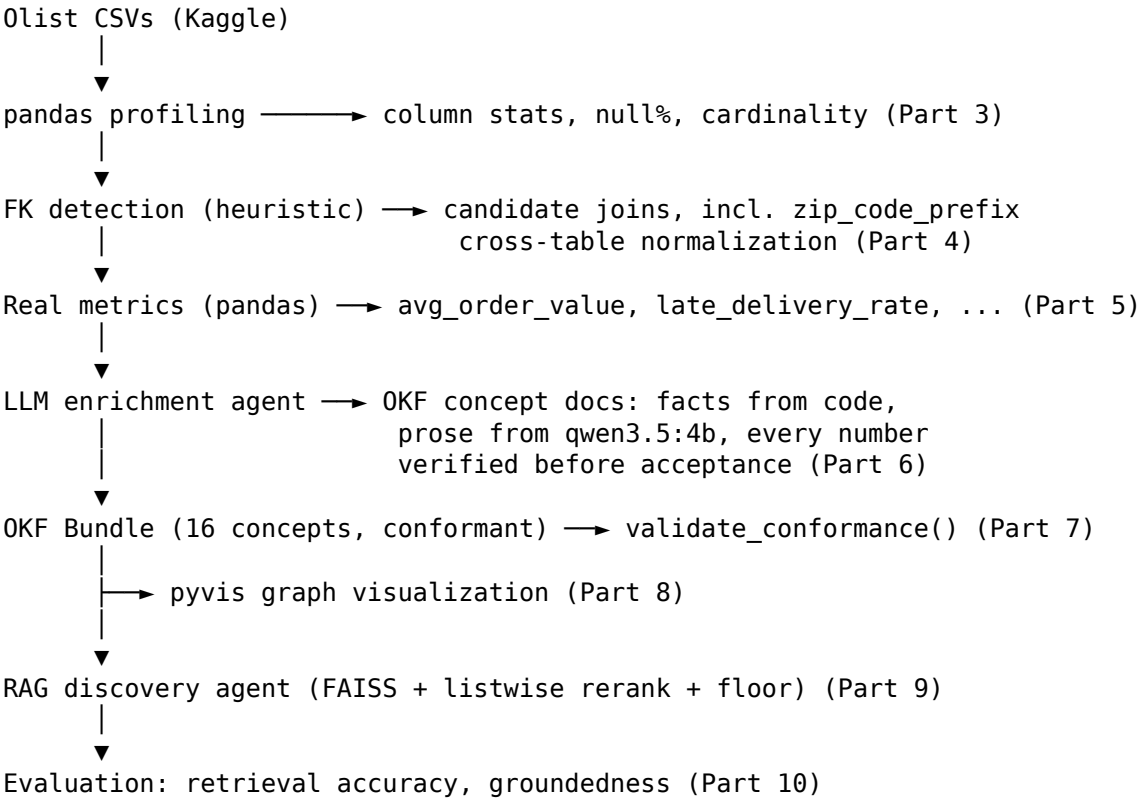
Part III — Notebook 1 Walkthrough: OKF From Scratch

File: `google_okf_zero_to_mastery.ipynb` · **Models**: `qwen3.5:4b` (generation, thinking disabled), `nomic-embed-text` (embeddings) · **Dataset**: Olist Brazilian E-Commerce Public Dataset

3.1 The real problem

The Olist dataset ships as nine CSV files with real foreign-key relationships documented nowhere in the files themselves — genuinely undocumented, genuinely production-shaped data. Notebook 1 acts as both **producer** (an enrichment agent drafting an OKF bundle, mirroring what Google's own reference agent does for BigQuery) and **consumer** (a retrieval agent answering real questions using only the bundle it produced).

3.2 Architecture



3.3 The OKF library, in outline

The core library (full code in Appendix A, Part 3) implements: `OkfDocument` (a dataclass for `type/title/description/resource/tags/timestamp/body`, with a `.render()` method producing spec-compliant frontmatter+body text), `parse_concept()` (the inverse — parse a file back into frontmatter dict + body string, raising if the frontmatter block doesn't parse), link extraction and resolution (classifying links as bundle-absolute/relative/external and resolving them to concept IDs), `generate_index()`/`generate_log()` (implementing SPEC.md §6/§7), and `validate_conformance()` (implementing SPEC.md §9's exact two hard rules plus soft link-checking).

3.4 Grounded generation, concretely

Every fact a concept document states is either computed in Python or verified after generation. For a table concept, the `# Schema` and `# Joins` sections are built entirely programmatically from `profile_table()` and `detect_fk_candidates()` output; the LLM only supplies the lead description and a short notes paragraph. For a metric concept, the real computed value is handed to the model with an explicit instruction to include it verbatim; `metric_is_grounded()` checks the generated text for a matching numeric substring, and a failed check triggers a stricter retry, then a deterministic templated fallback if the retry also fails. This pattern — compute the fact, ask for prose, verify the prose contains the fact — is the single most load-bearing idea in this notebook, and it caught two real hallucinations during development (Part VI).

3.5 Consumption: the discovery agent

All 16 concepts are embedded (`nomic-embed-text`) and indexed in a FAISS `IndexFlatIP`. A query is embedded, the top-k nearest concepts retrieved, and a listwise LLM rerank call asked to reorder them — with a **guaranteed floor**: the top `k_raw_guaranteed` raw retrieval hits always survive into the final context regardless of what the rerank call does, because an earlier version without this floor was measured to drop the single most relevant document for one question (Part VI, Bug #1). The final context is handed to a generation call that must cite concept IDs and must decline rather than guess if the context doesn't contain the answer.

3.6 Real results

- **16 conformant OKF concepts** (1 dataset, 9 tables, 6 metrics); 0 conformance errors, 0 warnings.
- **Retrieval accuracy** (8 hand-labeled questions): Hit@1 88%, Hit@3/Hit@5 100%.
- **Answer groundedness** (3 numeric questions): 3/3 — every generated answer contains the true computed number.

Part IV — Notebook 2 Walkthrough: Agentic RAG + ChromaDB

File: `agentic_rag_chromadb.ipynb` · **Models**: identical to notebook 1 · **Dataset**: identical to notebook 1 · **Key difference**: no OKF layer at all.

4.1 The alternative being tested

Same problem, same dataset, same models, same grounding discipline (including both hallucination guards notebook 1 discovered it needed) — but the *output* of enrichment is a flat `{id, text, metadata}` dict, not an OKF concept: no frontmatter, no bundle hierarchy, no parseable cross-links. Document IDs reuse notebook 1's naming (`tables/orders`, `references/metrics/avg_order_value`, …) purely as opaque string identifiers, kept only so the same hand-labeled eval set applies unchanged.

4.2 Architecture

```
Olist CSVs (Kaggle) ──► pandas profiling, FK detection, metrics (identical to notebook 1)
      │
      ▼
Flat document generation ──► {id, text, metadata} — no frontmatter, no links
      │
      ▼
ChromaDB (EphemeralClient, custom OllamaEmbeddingFunction)
      │
      ├──► Simple RAG: one query() call → generate  (the fair control)
      │
      └──► Agentic RAG: native Ollama tool-calling loop
                search_knowledge_base(query, top_k) — model decides
                whether/how many times to call it, ≤4 iterations
      │
      ▼
Benchmark: same 14-question eval set as the retired advanced-RAG notebook
```

4.3 Ingesting into ChromaDB

A `chromadb.EmbeddingFunction` subclass wraps `ollama.embed()` — subclassing (not duck-typing) is required, because Chroma's `EmbeddingFunction` protocol supplies a default `embed_query()` method that calls `self.__call__()`; a bare object missing that inheritance fails at query time with `AttributeError: ... no attribute 'embed_query'` (a real error hit and fixed during development, Part VI, Bug #3). An `EphemeralClient` (in-memory, no persistence) is used consistently with this project's "always rebuild fresh" pattern.

4.4 The agentic loop

The model is given one tool, `search_knowledge_base(query, top_k)`, and a system prompt explaining it may call the tool more than once if the first results are insufficient. Each loop iteration sends the conversation (including all prior tool results) back to the model; if the model's response contains no tool call, that response is the final answer. The loop is bounded at 4 iterations, after which a forced final-answer turn (no tools) is issued. Both the search query text and the returned document IDs are logged for every call, giving a complete, inspectable trace of what the agent actually did.

4.5 Real results

Metric	Simple RAG	Agentic RAG
Retrieval hit rate	91.7%	91.7%
Answer-citation hit rate	83.3%	75.0%
Refusal correctness (distractors)	100%	100%
Avg searches issued / question	1 (fixed)	1.17
Avg LLM+embed calls / question	2.0	3.29 (1.6×)

The number that matters most is 1.17. The agent almost never chose to search more than once — most questions were behaviorally identical to simple RAG. The exception was the multi-hop category (1.67 searches/question on average), where it won clearly (100% vs. 66.7% citation-hit): the one place genuinely needing more than one lookup is exactly where the agent chose to make more than one. Elsewhere (paraphrase: 66.7% vs. 100%) the extra autonomy bought nothing and mildly hurt. Zero of 14 questions hit the iteration cap.

Part V — OKF vs. RAG+VectorDB: Summary

A full, standalone treatment lives in `OKF_vs_RAG_Comparison.md/.pdf`. In brief: **OKF is what you retrieve *from*; RAG is *how* you retrieve it.** They are not competitors. OKF is a storage/authoring answer — durable, versioned, human-*and*-agent-curated, checkably well-formed. Flat RAG+VectorDB is a consumption-layer answer — fast to stand up over content nobody structured on purpose. This project needed the former (a dataset worth curating once) but built the latter anyway, specifically to make the tradeoff measured rather than asserted. The one empirical pattern that showed up twice, independently, in two different "smarter than naive" pipelines built on two different corpora (OKF-based advanced RAG, and flat-corpus agentic RAG): **added retrieval sophistication earns its keep specifically on multi-hop questions, and is neutral-to-harmful everywhere else** — see `OKF_vs_RAG_Comparison.md` §4.3 for the side-by-side category table.

Part VI — Bug Catalog: What Actually Broke, and Why

Every bug below was caught by actually running the code and checking real output — not by inspection — and every fix is live in the corresponding notebook. This section exists because the failure modes are more instructive than the successes.

Bug #1 — Listwise rerank silently dropped the correct document (Notebook 1)

Symptom: for "What columns does the orders table have…", dense retrieval correctly ranked `tables/orders` at position 2, but an unconstrained listwise LLM rerank call reordered it out of the top 3 entirely, replacing it with a less-relevant table. **Root cause**: `qwen3.5:4b` (4B, not distilled for reranking) asked to freely reorder 8 candidates in one call — exactly the regime the reranking literature (RankGPT/RankVicuna lineage) flags as unreliable for small, undistilled models. **Fix**: a guaranteed floor — the top `k_raw_guaranteed` raw retrieval hits always survive into final context regardless of the rerank call's output. Measured directly in notebook 1, Part 10.1.

Bug #2 — Confidently hallucinated date range (Notebook 1)

Symptom: an early version of the dataset-overview prompt gave the model row/column counts but no dates. It responded with a fluent, specific, and simply invented "between 2019 and 2023" — the real range is 2016-09-04 to 2018-10-17. **Root cause**: the prompt left a gap (no date information) and the model filled it plausibly rather than declining to guess. **Fix**: hand the model the real date range explicitly, and verify no other year appears in the output (`years_ok()`, with retry-then-deterministic-fallback) — the same pattern as metric groundedness checking, applied to dates.

Bug #3 — ChromaDB `AttributeError: no attribute 'embed_query'` (Notebook 2)

Symptom: a duck-typed embedding function class (matching `EmbeddingFunction`'s `__call__` signature without inheriting from it) worked for *adding* documents but failed at *query* time. **Root cause**: Chroma's `EmbeddingFunction` is a `Protocol` whose mixin `embed_query()` method (which defaults to calling `self.__call__()`) is only attached via actual subclassing — not present on a duck-typed lookalike. **Fix**: `class OllamaEmbeddingFunction(EmbeddingFunction):` — proper inheritance. Caught by a targeted smoke test before it could surface mid-notebook.

Bug #4 — Graph expansion was a structural no-op (retired advanced-RAG notebook)

Symptom: a graph-expansion stage was computed on every call but never actually influenced the final retrieved context. **Root cause**: the candidate pool was built as `(fused_ids + graph_ids)[:pool_cap]`; since `fused_ids` alone routinely exceeded `pool_cap`, the slice never reached far enough to include a single graph-expanded candidate. **Fix**: reserve a fixed number of pool slots for graph-expanded candidates up front, rather than appending them after an already-full list.

Bug #5 — Pointwise reranker judged relevance from too little context (retired advanced-RAG notebook)

Symptom: the pointwise judge, given only a one-line frontmatter `description`, sometimes ranked a topically-adjacent metric document above the table that directly answered a schema question. **Root cause**: a generic one-sentence summary doesn't obviously signal "this document has the columns you're asking about" the way the document's actual `# Schema` section

would. **Fix**: score on a real body excerpt (~500 characters of actual content) instead of the one-line description.

The pattern across all five: every one was caught because this project's default posture is to run the code and check the real output rather than assume a plausible-sounding design is a correct one — and every fix is a direct, mechanical response to what the real output showed, not a speculative hardening pass.

Part VII — Consolidated References

1. Open Knowledge Format v0.1 specification — SPEC.md, `GoogleCloudPlatform/knowledge-catalog`
2. Google Cloud Blog — "How the Open Knowledge Format can improve data sharing"
3. Cormack, Clarke & Büttcher (2009). "Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods." SIGIR '09.
4. Rackauckas (2024). "RAG-Fusion: a New Take on Retrieval-Augmented Generation."
5. Gao et al. (2022). "Precise Zero-Shot Dense Retrieval without Relevance Labels" (HyDE), ACL 2023.
6. Edge et al., Microsoft Research. "GraphRAG: Unlocking LLM discovery on narrative private data."
7. Sun et al. (2023). "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents" (RankGPT).
8. Yao et al. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models."
9. Singh et al. (2025). "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG."
10. ChromaDB documentation
11. `rank_bm25` — BM25Okapi implementation.
12. `pyvis` — graph visualization; FAISS — similarity search.
13. Ollama tool-calling — native function-calling API.
14. Dataset — Brazilian E-Commerce Public Dataset by Olist (Kaggle, CC BY-NC-SA 4.0).

Appendix A — Full Source: `google_okf_zero_to_mastery.ipynb`

Google Open Knowledge Format (OKF) — Zero to Mastery

```
import json
import os
import re
import subprocess
import time
import warnings
from collections import defaultdict
from dataclasses import dataclass, field
from datetime import datetime, timezone
from itertools import groupby
from pathlib import Path
from typing import Any, Optional

import faiss
import numpy as np
import ollama
import pandas as pd
import yaml
from loguru import logger
from tqdm.auto import tqdm

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 140)

GEN_MODEL = "qwen3.5:4b"
EMBED_MODEL = "nomic-embed-text"
BUNDLE_ROOT = Path("bundle")
RUN_TS = datetime.now(timezone.utc).replace(microsecond=0).isoformat()

logger.remove()
logger.add(lambda msg: print(msg, end=""), level="INFO",
 ↪  format="<level>{message}</level>\n")
```

```
local_models = {m["model"] for m in ollama.list()["models"]}


def model_available(name: str) -> bool:
    # `ollama list` returns tagged names (e.g. "nomic-embed-text:latest"); match
     ↪  with or without a tag.
    return name in local_models or any(m.split(":")[0] == name.split(":")[0] for m
     ↪  in local_models)


for required in (GEN_MODEL, EMBED_MODEL):
    assert model_available(required), (
        f"Missing local model {required!r}. Pull it with `ollama pull {required}`
         ↪  first."
    )
```

```
print("Available and ready:")
print(f"  generation model : {GEN_MODEL}")
print(f"  embedding model  : {EMBED_MODEL}")
print(f"  run timestamp    : {RUN_TS}")
```

```
def llm(prompt: str, *, system: Optional[str] = None, num_predict: int = 350,
        temperature: float = 0.2, retries: int = 2) -> str:
    """Call the local generation model with thinking disabled (qwen3.5 is a
     ↪  hybrid-reasoning
    model; without `think=False` it can burn its entire token budget on hidden
 ↪  <think> traces
    and return empty content — verified during development of this notebook).
    """
    messages = []
    if system:
        messages.append({"role": "system", "content": system})
    messages.append({"role": "user", "content": prompt})
    last_err = None
    for attempt in range(retries + 1):
        try:
            resp = ollama.chat(
                model=GEN_MODEL,
                messages=messages,
                think=False,
                options={"num_predict": num_predict, "temperature": temperature},
            )
            content = resp["message"]["content"].strip()
            if content:
                return content
        except Exception as e:  # pragma: no cover - network/runtime hiccups
            last_err = e
        time.sleep(0.5)
    raise RuntimeError(f"LLM call failed after {retries + 1} attempts: {last_err}")


def embed(text: str) -> np.ndarray:
    resp = ollama.embed(model=EMBED_MODEL, input=text)
    return np.array(resp["embeddings"][0], dtype="float32")


# smoke test
print(llm("Reply with exactly the word: ready"))
```

0. Environment check

```
mini_example = """---
type: Metric
title: Average Order Value
description: Mean total payment value per order.
tags: [metrics, revenue]
timestamp: 2026-01-01T00:00:00+00:00
---
```

```
The average order value is the mean of total payments across all orders.

# Citations

[1] Computed from the dataset in this notebook.
"""

fm_block, body = re.match(r"^---\n(.*?)\n---\n(.*)$", mini_example,
 ↪  re.DOTALL).groups()
frontmatter = yaml.safe_load(fm_block)
print("Parsed frontmatter:", frontmatter)
assert frontmatter.get("type"), "Rule §9.2 violated: 'type' must be non-empty"
print("\nConformant: type is present and non-empty ->",
 ↪  bool(frontmatter.get("type")))
```

1. Theory — What is the Open Knowledge Format?

```
import kagglehub

DATASET_DIR = Path(kagglehub.dataset_download("olistbr/brazilian-ecommerce"))
csv_files = sorted(DATASET_DIR.glob("*.csv"))
print(f"Downloaded to: {DATASET_DIR}")
for f in csv_files:
    print(f"  {f.name:45s} {f.stat().st_size / 1e6:6.2f} MB")
```

2. The real problem

```
TABLE_RENAME = {"product_category_name_translation": "category_translation"}


def table_name_from_file(path: Path) -> str:
    name = path.stem
    name = re.sub(r"^olist_", "", name)
    name = re.sub(r"_dataset$", "", name)
    return TABLE_RENAME.get(name, name)


tables: dict[str, pd.DataFrame] = {}
for f in csv_files:
    name = table_name_from_file(f)
    tables[name] = pd.read_csv(f)

print(f"Loaded {len(tables)} tables:\n")
for name, df in tables.items():
    print(f"  {name:35s} {df.shape[0]:>8,d} rows  x {df.shape[1]:>2d} cols")
```

2.2 Load every table

```
def profile_table(name: str, df: pd.DataFrame) -> dict:
```

```
    columns = []
    for col in df.columns:
        s = df[col]
        samples = [str(v) for v in s.dropna().unique()[:3]]
        columns.append({
            "name": col,
            "dtype": str(s.dtype),
            "null_pct": round(float(s.isna().mean()) * 100, 2),
            "n_unique": int(s.nunique()),
            "samples": samples,
        })
    return {"name": name, "n_rows": int(len(df)), "n_cols": int(len(df.columns)),
     ↪  "columns": columns}


profiles = {name: profile_table(name, df) for name, df in tables.items()}

overview_rows = []
for name, p in profiles.items():
    overview_rows.append({
        "table": name,
        "rows": p["n_rows"],
        "cols": p["n_cols"],
        "avg_null_pct": round(np.mean([c["null_pct"] for c in p["columns"]]), 2),
    })
display(pd.DataFrame(overview_rows).sort_values("rows",
 ↪  ascending=False).reset_index(drop=True))
```

```
# A closer look at one table, to see exactly what the enrichment agent will be
 ↪  grounded on.
pd.DataFrame(profiles["orders"]["columns"])
```

2.3 Real EDA — schema, nulls, and samples for every table

```
RESERVED_FILENAMES = {"index.md", "log.md"}


@dataclass
class OkfDocument:
    """One OKF concept document: a required `type`, recommended metadata, and a
     ↪  free-form body."""

    type: str
    title: Optional[str] = None
    description: Optional[str] = None
    resource: Optional[str] = None
    tags: list[str] = field(default_factory=list)
    timestamp: Optional[str] = None
    extra: dict[str, Any] = field(default_factory=dict)
    body: str = ""

    def frontmatter_dict(self) -> dict:
```

```
        assert self.type, "SPEC.md §9.2: 'type' is a required, non-empty
         ↪  frontmatter field"
        fm: dict[str, Any] = {"type": self.type}
        if self.title:
            fm["title"] = self.title
        if self.description:
            fm["description"] = self.description
        if self.resource:
            fm["resource"] = self.resource
        if self.tags:
            fm["tags"] = self.tags
        if self.timestamp:
            fm["timestamp"] = self.timestamp
        fm.update(self.extra)
        return fm

    def render(self) -> str:
        fm_yaml = yaml.safe_dump(
            self.frontmatter_dict(), sort_keys=False, allow_unicode=True,
             ↪  default_flow_style=False
        ).strip()
        return f"---\n{fm_yaml}\n---\n\n{self.body.strip()}\n"


FRONTMATTER_RE = re.compile(r"^---\n(.*?)\n---\n(.*)$", re.DOTALL)


def parse_concept(path: Path) -> tuple[dict, str]:
    """Parse a concept file into (frontmatter dict, body). Raises if no parseable
     ↪  block exists —
    this is the one hard structural rule a *producer* must satisfy (SPEC.md
 ↪  §9.1)."""
    text = path.read_text(encoding="utf-8")
    m = FRONTMATTER_RE.match(text)
    if not m:
        raise ValueError(f"{path}: no parseable YAML frontmatter block (SPEC.md
         ↪  §9.1 violation)")
    fm = yaml.safe_load(m.group(1)) or {}
    return fm, m.group(2)


def write_concept(bundle_root: Path, concept_id: str, doc: OkfDocument) -> Path:
    if Path(f"{concept_id}.md").name in RESERVED_FILENAMES:
        raise ValueError(f"{concept_id!r} collides with a reserved filename")
    path = bundle_root / f"{concept_id}.md"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(doc.render(), encoding="utf-8")
    return path


def iter_concept_files(bundle_root: Path):
    for p in sorted(bundle_root.rglob("*.md")):
        if p.name not in RESERVED_FILENAMES:
            yield p
```

```
_test_dir = Path("_okf_selftest")
_test_doc = OkfDocument(
    type="Metric", title="Test Metric", description="A round-trip test.",
    tags=["test"], timestamp=RUN_TS, body="# Citations\n\n[1] N/A",
)
_test_path = write_concept(_test_dir, "metrics/test", _test_doc)
_fm, _body = parse_concept(_test_path)
assert _fm["type"] == "Metric" and _fm["title"] == "Test Metric" and "Citations"
 ↪  in _body
print("Round-trip OK:", _fm)
import shutil
shutil.rmtree(_test_dir)
```

3. The OKF core library — built from scratch

```
LINK_RE = re.compile(r"\[([^\]]+)\]\(([^)]+)\)")


def extract_links(body: str) -> list[dict]:
    links = []
    for text, target in LINK_RE.findall(body):
        if target.startswith(("http://", "https://")):
            kind = "external"
        elif target.startswith("/"):
            kind = "bundle-absolute"
        else:
            kind = "relative"
        links.append({"text": text, "target": target, "kind": kind})
    return links


def concept_id_for(path: Path, bundle_root: Path) -> str:
    return str(path.relative_to(bundle_root).with_suffix("")).replace(os.sep, "/")


def resolve_link_target(link: dict, from_concept_id: str) -> Optional[str]:
    """Best-effort resolution of a link to a concept ID, for graph-building and
     ↪  link-checking."""
    if link["kind"] == "external":
        return None
    if link["kind"] == "bundle-absolute":
        target = link["target"].lstrip("/")
    else:  # relative
        target = str((Path(from_concept_id).parent / link["target"]).as_posix())
    return re.sub(r"\.md$", "", target)
```

3.2 Cross-linking (SPEC.md §5)

```
def generate_index(dir_path: Path, bundle_root: Path, *, okf_version:
 ↪  Optional[str] = None,
                   title: Optional[str] = None) -> str:
```

```
    groups: dict[str, list[tuple[str, str, str]]] = defaultdict(list)
    for child in sorted(dir_path.iterdir()):
        if child.name in RESERVED_FILENAMES:
            continue
        if child.is_dir():
            groups["Subdirectories"].append((child.name + "/", child.name + "/",
↪   f"{child.name} concepts"))
        elif child.suffix == ".md":
            fm, _ = parse_concept(child)
            child_title = fm.get("title", child.stem)
            desc = fm.get("description", "")
            groups[fm.get("type", "Concept")].append((child_title, child.name,
↪   desc))

    lines = []
    if okf_version:
        lines += ["---", f"okf_version: \"{okf_version}\"", "---", ""]
    lines.append(f"# {title or dir_path.name or bundle_root.name}\n")
    for group, entries in groups.items():
        lines.append(f"# {group}\n")
        for title, href, desc in entries:
            lines.append(f"* [{title}]({href})" + (f" - {desc}" if desc else ""))
        lines.append("")
    return "\n".join(lines).strip() + "\n"


def write_index(dir_path: Path, bundle_root: Path, **kwargs) -> Path:
    text = generate_index(dir_path, bundle_root, **kwargs)
    path = dir_path / "index.md"
    path.write_text(text, encoding="utf-8")
    return path
```

3.3 Index files (SPEC.md §6) — progressive disclosure

```
def generate_log(entries: list[dict]) -> str:
    lines = ["# Directory Update Log\n"]
    entries = sorted(entries, key=lambda e: e["date"], reverse=True)
    for date, group in groupby(entries, key=lambda e: e["date"]):
        lines.append(f"## {date}")
        for e in group:
            lines.append(f"* **{e['kind']}**: {e['text']}")
        lines.append("")
    return "\n".join(lines).strip() + "\n"
```

3.4 Log files (SPEC.md §7)

```
def validate_conformance(bundle_root: Path) -> dict:
    errors, warnings_, concept_ids = [], [], []
    parsed: dict[str, tuple[dict, str]] = {}

    for p in iter_concept_files(bundle_root):
        cid = concept_id_for(p, bundle_root)
```

```
        concept_ids.append(cid)
        try:
            fm, body = parse_concept(p)
        except ValueError as e:
            errors.append(str(e))
            continue
        if not fm.get("type"):
            errors.append(f"{cid}: missing/empty required 'type' field (SPEC.md
↪   §9.2)")
        parsed[cid] = (fm, body)

    id_set = set(concept_ids)
    for cid, (fm, body) in parsed.items():
        for link in extract_links(body):
            target = resolve_link_target(link, cid)
            if target is not None and target not in id_set:
                warnings_.append(f"{cid}: broken link -> {link['target']}
↪   (tolerated per SPEC.md §9)")

    for reserved in RESERVED_FILENAMES:
        for p in bundle_root.rglob(reserved):
            if reserved == "index.md" and p.read_text(encoding="utf-8").strip() ==
             ↪  "":
                warnings_.append(f"{p.relative_to(bundle_root)}: empty index.md")

    return {
        "conformant": len(errors) == 0,
        "n_concepts": len(concept_ids),
        "n_errors": len(errors),
        "n_warnings": len(warnings_),
        "errors": errors,
        "warnings": warnings_,
    }
```

3.5 Conformance validator (SPEC.md §9)

```
def base_key(col: str) -> str:
    return "zip_code_prefix" if col.endswith("zip_code_prefix") else col


def detect_fk_candidates(tables: dict[str, pd.DataFrame], min_overlap: float =
↪   0.9) -> pd.DataFrame:
    groups: dict[str, list[tuple[str, str]]] = defaultdict(list)
    for tname, df in tables.items():
        for col in df.columns:
            groups[base_key(col)].append((tname, col))

    rows = []
    for members in groups.values():
        if len(members) < 2:
            continue
        for i, (t1, c1) in enumerate(members):
            for j, (t2, c2) in enumerate(members):
```

```
                if i == j:
                    continue
                s1, s2 = tables[t1][c1].dropna(), tables[t2][c2].dropna()
                if s1.empty or s2.empty:
                    continue
                overlap = float(s1.isin(set(s2.unique())).mean())
                if overlap >= min_overlap:
                    rows.append({
                        "from_table": t1, "from_column": c1, "to_table": t2,
                        ↪  "to_column": c2,
                        "overlap": round(overlap, 4),
                        "from_cardinality": int(s1.nunique()), "to_cardinality":
                        ↪  int(s2.nunique()),
                    })
    return pd.DataFrame(rows)


fk_candidates = detect_fk_candidates(tables)
display(fk_candidates.sort_values(["from_table", "overlap"], ascending=[True,
 ↪  False]).reset_index(drop=True))
```

4. Foreign-key discovery — deterministic, not the LLM's job

```
orders_df = tables["orders"].copy()
for col in ["order_purchase_timestamp", "order_delivered_customer_date",
 ↪  "order_estimated_delivery_date"]:
    orders_df[col] = pd.to_datetime(orders_df[col], errors="coerce")

payments_per_order =
 ↪  tables["order_payments"].groupby("order_id")["payment_value"].sum()
avg_order_value = float(payments_per_order.mean())

delivered = orders_df[orders_df["order_status"] == "delivered"].dropna(
    subset=["order_delivered_customer_date", "order_estimated_delivery_date"]
)
late_mask = delivered["order_delivered_customer_date"] >
 ↪  delivered["order_estimated_delivery_date"]
late_delivery_rate = float(late_mask.mean())

review_scores = tables["order_reviews"]["review_score"]
avg_review_score = float(review_scores.mean())
review_score_distribution =
 ↪  (review_scores.value_counts(normalize=True).sort_index() * 100).round(2)

payment_type_distribution = (
    tables["order_payments"]["payment_type"].value_counts(normalize=True) * 100
).round(2)

cat_en = tables["products"].merge(tables["category_translation"],
 ↪  on="product_category_name", how="left")
top_categories = (
    tables["order_items"]
    .merge(cat_en[["product_id", "product_category_name_english"]],
 ↪  on="product_id", how="left")
```

```
    .groupby("product_category_name_english")["order_id"].nunique()
    .sort_values(ascending=False).head(5)
)

metrics_summary = {
    "avg_order_value": {
        "value": round(avg_order_value, 2), "unit": "BRL",
        "n": int(payments_per_order.shape[0]),
        "source_tables": ["order_payments"],
        "definition": "mean(sum(payment_value) grouped by order_id)",
    },
    "late_delivery_rate": {
        "value": round(late_delivery_rate * 100, 2), "unit": "%",
        "n": int(len(delivered)),
        "source_tables": ["orders"],
        "definition": "mean(order_delivered_customer_date >
        ↪  order_estimated_delivery_date) over delivered orders",
    },
    "avg_review_score": {
        "value": round(avg_review_score, 2), "unit": "stars (1-5)",
        "n": int(review_scores.notna().sum()),
        "source_tables": ["order_reviews"],
        "definition": "mean(review_score)",
    },
    "review_score_distribution": {
        "value": review_score_distribution.to_dict(), "unit": "% of reviews",
        "n": int(review_scores.notna().sum()),
        "source_tables": ["order_reviews"],
        "definition": "value_counts(review_score, normalize=True)",
    },
    "payment_type_distribution": {
        "value": payment_type_distribution.to_dict(), "unit": "% of payments",
        "n": int(tables["order_payments"].shape[0]),
        "source_tables": ["order_payments"],
        "definition": "value_counts(payment_type, normalize=True)",
    },
    "top_product_categories": {
        "value": top_categories.to_dict(), "unit": "distinct orders",
        "n": int(top_categories.sum()),
        "source_tables": ["order_items", "products", "category_translation"],
        "definition": "nunique(order_id) grouped by product_category_name_english,
        ↪  top 5",
    },
}

for k, v in metrics_summary.items():
    print(f"{k:28s} = {v['value']}  {v['unit']}  (n={v['n']:,})")
```

5. Real metrics — computed once, cited everywhere

```
KAGGLE_URL = "https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce"
```

```
def format_schema_table(profile: dict) -> str:
    lines = ["| Column | Type | Null % | Unique | Sample values |",
 ↪  "|---|---|---|---|---|"]
    for c in profile["columns"]:
        samples = ", ".join(f"`{s[:30]}`" for s in c["samples"]) or "*(none)*"
        lines.append(f"| `{c['name']}` | {c['dtype']} | {c['null_pct']}% |
 ↪  {c['n_unique']:,} | {samples} |")
    return "\n".join(lines)


def format_joins_section(table: str, fk_df: pd.DataFrame) -> str:
    outgoing = fk_df[fk_df["from_table"] == table]
    if outgoing.empty:
        return "No outgoing foreign-key relationships were detected from this
         ↪  table."
    lines = []
    for _, r in outgoing.iterrows():
        lines.append(
            f"- `{r['from_column']}` ->
             ↪  [{r['to_table']}](/tables/{r['to_table']}.md) "
            f"(`{r['overlap'] * 100:.1f}%` value overlap, "
           f"{r['from_cardinality']:,} -> {r['to_cardinality']:,} distinct values)"
        )
    return "\n".join(lines)


def first_sentence(text: str, max_len: int = 220) -> str:
    """Frontmatter `description` should be a clean one-liner. Blind character
     ↪  slicing cuts mid-word
    (a real bug caught during development: '...capturing end-to-end data from
 ↪  produc' — truncated
    mid-"product"); breaking on a sentence boundary first avoids that."""
    text = text.strip()
    m = re.search(r".+?[.!?](?=\s|$)", text)
    sentence = m.group(0) if m else text
    if len(sentence) > max_len:
        sentence = sentence[:max_len].rsplit(" ", 1)[0].rstrip(".,;:") + "…"
    return sentence


def format_metric_value(info: dict) -> str:
    v = info["value"]
    if isinstance(v, dict):
        return ", ".join(f"{k}: {v2}" for k, v2 in v.items())
    return str(v)


def number_variants(x: float) -> set[str]:
    return {f"{x}", f"{x:.0f}", f"{x:.1f}", f"{x:.2f}", str(round(x))}


def metric_is_grounded(text: str, info: dict) -> bool:
    """A generated metric explanation is 'grounded' if at least one of the real
     ↪  computed numbers
```

```
    appears verbatim in the text. This is a cheap but real faithfulness check —
 ↪  exactly the kind
    of thing that matters once an LLM is writing documentation nobody will manually
 ↪  fact-check."""
    clean = text.replace(",", "")
    v = info["value"]
    candidates = v.values() if isinstance(v, dict) else [v]
    return any(variant in clean for x in candidates for variant in
     ↪  number_variants(float(x)))
```

6. The local LLM enrichment agent

```
def build_table_prompt(name: str, profile: dict, fk_df: pd.DataFrame) -> str:
    outgoing = fk_df[fk_df["from_table"] == name]
    incoming = fk_df[fk_df["to_table"] == name]
    rel_bits = [f"{name}.{r.from_column} -> {r.to_table}.{r.to_column} ({r.overlap
 ↪  * 100:.0f}% overlap)"
                for r in outgoing.itertuples()]
    rel_bits += [f"{r.from_table}.{r.from_column} -> {name}.{r.to_column}
 ↪  ({r.overlap * 100:.0f}% overlap)"
                 for r in incoming.itertuples()]
    col_summary = "; ".join(f"{c['name']} ({c['dtype']}, {c['null_pct']}% null)"
 ↪  for c in profile["columns"])
    return f"""You are documenting a database table for a knowledge base read by
     ↪  engineers and AI agents.
Use ONLY the facts given below. Do not invent business meaning beyond what the
 ↪  column names imply, and never invent numbers.

Table name: {name}
Row count: {profile['n_rows']:,}
Columns: {col_summary}
Detected relationships: {'; '.join(rel_bits) if rel_bits else 'none detected'}

Write exactly two short paragraphs separated by a single blank line:
1. A 1-2 sentence description of what one row in this table represents.
2. A 1-2 sentence note mentioning any relationships to other tables and any
 ↪  notable data-quality
   signal (e.g. a column with a high null percentage), using only the facts above.

Output ONLY the two paragraphs as plain prose. No headers, no markdown, no
 ↪  preamble."""


def build_metric_prompt(key: str, info: dict) -> str:
    value_str = format_metric_value(info)
    return f"""You are documenting a business metric for a knowledge base.

Metric: {key.replace('_', ' ')}
Computed value: {value_str} {info['unit']}
Computed over: {info['n']:,} records
Definition: {info['definition']}
Source table(s): {', '.join(info['source_tables'])}
```

```
Write a 2-3 sentence plain-English explanation of what this metric measures and
 ↪  what its computed
value means in practice. You MUST include the exact computed value ({value_str})
 ↪  verbatim somewhere
in your text. Output ONLY the prose — no headers, no preamble."""


def build_dataset_prompt(profiles: dict, date_range: str) -> str:
    """Includes the real order date range explicitly. Without it, a first run of
     ↪  this exact prompt
    produced a confident, plausible-sounding, and completely wrong '...between
 ↪  2019 and 2023' —
    the model filling a gap we hadn't grounded. Handing it the true range
 ↪  (2016-09-04 to
    2018-10-17) removes the incentive to guess."""
    table_list = "\n".join(f"- {n}: {p['n_rows']:,} rows, {p['n_cols']} columns"
 ↪  for n, p in profiles.items())
    return f"""You are documenting an entire dataset for a knowledge base.

Dataset: Olist Brazilian E-Commerce Public Dataset
Order date range: {date_range}
Tables:
{table_list}

Write a 2-3 sentence overview of what this dataset represents as a whole and what
 ↪  kind of business
questions it can answer. Use ONLY the facts given above — in particular, do not
 ↪  state or imply any
date range other than the one given. Output ONLY the prose, no headers, no
 ↪  preamble."""
```

6.2 Prompt templates

```
def enrich_table(name: str) -> OkfDocument:
    profile = profiles[name]
    raw = llm(build_table_prompt(name, profile, fk_candidates), num_predict=250)
    parts = raw.split("\n\n", 1)
    description = parts[0].strip()
    notes = parts[1].strip() if len(parts) > 1 else "No additional notes
 ↪  generated."
    body = f"""{description}

# Schema

{format_schema_table(profile)}

# Joins

{format_joins_section(name, fk_candidates)}

# Notes

{notes}
```

```
# Citations

[1] [Olist Brazilian E-Commerce Public Dataset (Kaggle)]({KAGGLE_URL}) — table
 ↪  `{name}`."""
    return OkfDocument(
        type="Table", title=name.replace("_", " ").title(),
         ↪  description=first_sentence(description),
        resource=KAGGLE_URL, tags=["olist", "ecommerce", name], timestamp=RUN_TS,
 ↪  body=body,
    )


def enrich_metric(key: str) -> OkfDocument:
    info = metrics_summary[key]
    prompt = build_metric_prompt(key, info)
    text = llm(prompt, num_predict=200)
    if not metric_is_grounded(text, info):
        logger.warning(f"{key}: ungrounded on first attempt, retrying with a
 ↪  stricter reminder")
        text = llm(prompt + f"\n\nReminder: the number {format_metric_value(info)}
 ↪  MUST appear verbatim.",
                   num_predict=200)
    if not metric_is_grounded(text, info):
        logger.warning(f"{key}: still ungrounded after retry — falling back to a
 ↪  deterministic template")
        text = (f"The {key.replace('_', ' ')} is {format_metric_value(info)},
 ↪  computed as "
                f"{info['definition']} over {info['n']:,} records from {',
                 ↪  '.join(info['source_tables'])}.")
    body = f"""{text}

# Definition

`{info['definition']}`

# Computed Value

- **Value**: {format_metric_value(info)} {info['unit']}
- **Computed over**: {info['n']:,} records
- **Source table(s)**: {", ".join(f"[{t}](/tables/{t}.md)" for t in
 ↪  info['source_tables'])}

# Citations

[1] Computed directly from the Olist dataset in this notebook (see Part 5)."""
    return OkfDocument(
        type="Metric", title=key.replace("_", " ").title(),
         ↪  description=first_sentence(text),
        tags=["metric", "olist"], timestamp=RUN_TS, body=body,
    )


def enrich_dataset() -> OkfDocument:
```

```
    date_min, date_max = orders_df["order_purchase_timestamp"].min(),
↪   orders_df["order_purchase_timestamp"].max()
    date_range = f"{date_min:%Y-%m-%d} to {date_max:%Y-%m-%d}"
    valid_years = {date_min.year, date_max.year}

    def years_are_grounded(t: str) -> bool:
        mentioned = {int(y) for y in re.findall(r"\b(20\d{2})\b", t)}
        return mentioned.issubset(valid_years)

    prompt = build_dataset_prompt(profiles, date_range)
    text = llm(prompt, num_predict=200)
    if not years_are_grounded(text):
        logger.warning(f"dataset overview mentioned a year outside
↪   {sorted(valid_years)}, retrying")
        text = llm(prompt + f"\n\nReminder: the only valid years are
↪   {sorted(valid_years)}.", num_predict=200)
    if not years_are_grounded(text):
        logger.warning("dataset overview still ungrounded on date range after retry
↪   — falling back to a deterministic template")
        text = (f"The Olist Brazilian E-Commerce Public Dataset captures orders
↪   placed between "
                f"{date_min:%B %Y} and {date_max:%B %Y} across {len(profiles)}
                 ↪  relational tables "
                f"spanning products, customers, sellers, payments, and reviews.")

    table_list = "\n".join(f"- [{n}](/tables/{n}.md) - {p['n_rows']:,} rows" for
↪   n, p in profiles.items())
    body = f"""{text}

# Tables

{table_list}

# Citations

[1] [Olist Brazilian E-Commerce Public Dataset (Kaggle)]({KAGGLE_URL})"""
    return OkfDocument(
        type="Dataset", title="Olist Brazilian E-Commerce",
         ↪  description=first_sentence(text),
        resource=KAGGLE_URL, tags=["olist", "ecommerce"], timestamp=RUN_TS,
↪   body=body,
    )
```

6.3 The three enrichment functions

```
import shutil

if BUNDLE_ROOT.exists():
    shutil.rmtree(BUNDLE_ROOT)

log_entries = []
run_date = RUN_TS[:10]
```

```
logger.info("Enriching dataset-level concept...")
write_concept(BUNDLE_ROOT, "datasets/olist_ecommerce", enrich_dataset())
log_entries.append({"date": run_date, "kind": "Creation", "text": "Created
 ↪  dataset-level concept `datasets/olist_ecommerce`."})

logger.info(f"Enriching {len(profiles)} table concepts...")
for name in tqdm(list(profiles), desc="tables"):
    write_concept(BUNDLE_ROOT, f"tables/{name}", enrich_table(name))
    log_entries.append({"date": run_date, "kind": "Creation", "text": f"Created
 ↪  table concept `tables/{name}`."})

logger.info(f"Enriching {len(metrics_summary)} metric concepts...")
for key in tqdm(list(metrics_summary), desc="metrics"):
    write_concept(BUNDLE_ROOT, f"references/metrics/{key}", enrich_metric(key))
    log_entries.append({"date": run_date, "kind": "Creation", "text": f"Created
 ↪  metric concept `references/metrics/{key}`."})

n_written = len(list(iter_concept_files(BUNDLE_ROOT)))
print(f"\nWrote {n_written} concept documents to '{BUNDLE_ROOT}/'")
```

```
write_index(BUNDLE_ROOT / "tables", BUNDLE_ROOT, title="Tables")
write_index(BUNDLE_ROOT / "datasets", BUNDLE_ROOT, title="Datasets")
write_index(BUNDLE_ROOT / "references" / "metrics", BUNDLE_ROOT, title="Metrics")
write_index(BUNDLE_ROOT / "references", BUNDLE_ROOT, title="References")
write_index(BUNDLE_ROOT, BUNDLE_ROOT, okf_version="0.1", title="Olist Brazilian
 ↪  E-Commerce — OKF Knowledge Bundle")
(BUNDLE_ROOT / "log.md").write_text(generate_log(log_entries), encoding="utf-8")

print((BUNDLE_ROOT / "index.md").read_text())
```

6.4 Run the agent and assemble the bundle

```
report = validate_conformance(BUNDLE_ROOT)
print(json.dumps({k: v for k, v in report.items() if k not in ("errors",
 ↪  "warnings")}, indent=2))

if report["warnings"]:
    print(f"\n{len(report['warnings'])} warning(s) — tolerated per SPEC.md §9, not
     ↪  conformance failures:")
    for w in report["warnings"]:
        print(" -", w)
if report["errors"]:
    print(f"\n{len(report['errors'])} error(s):")
    for e in report["errors"]:
        print(" -", e)

assert report["conformant"], "Bundle is NOT OKF v0.1 conformant"
print("\nResult: CONFORMANT — every concept has a parseable frontmatter block with
 ↪  a non-empty 'type'.")
```

7. Conformance validation (SPEC.md §9)

```
from pyvis.network import Network

TYPE_COLORS = {"Dataset": "#4C6EF5", "Table": "#12B886", "Metric": "#F59F00"}


def build_graph(bundle_root: Path) -> Network:
    net = Network(height="750px", width="100%", directed=True, notebook=False,
 ↪  cdn_resources="in_line")
    parsed = {concept_id_for(p, bundle_root): parse_concept(p) for p in
 ↪  iter_concept_files(bundle_root)}
    id_set = set(parsed)

    for cid, (fm, _) in parsed.items():
        net.add_node(
            cid, label=fm.get("title", cid), title=fm.get("description", ""),
            color=TYPE_COLORS.get(fm.get("type"), "#868E96"),
        )
    for cid, (_, body) in parsed.items():
        for link in extract_links(body):
            target = resolve_link_target(link, cid)
            if target in id_set and target != cid:
                net.add_edge(cid, target)
    return net


graph = build_graph(BUNDLE_ROOT)
viz_path = BUNDLE_ROOT / "viz.html"
graph.write_html(str(viz_path), open_browser=False)
print(f"Wrote interactive graph to '{viz_path}' ({viz_path.stat().st_size /
 ↪  1e3:.0f} KB, self-contained).")
```

```
from IPython.display import IFrame

IFrame(src=str(viz_path), width="100%", height=650)
```

8. Visualization — the bundle as a graph

```
def load_bundle_concepts(bundle_root: Path) -> list[dict]:
    out = []
    for p in iter_concept_files(bundle_root):
        cid = concept_id_for(p, bundle_root)
        fm, body = parse_concept(p)
        out.append({"id": cid, "type": fm.get("type"), "title": fm.get("title",
 ↪  cid),
                    "description": fm.get("description", ""), "body": body})
    return out


concepts = load_bundle_concepts(BUNDLE_ROOT)
concept_by_id = {c["id"]: c for c in concepts}
print(f"Loaded {len(concepts)} concepts for retrieval.")
```

```
def embed_text_for(c: dict) -> str:
    return f"{c['title']}\n{c['description']}\n{c['body'][:1500]}"


embeddings = np.stack([embed(embed_text_for(c)) for c in tqdm(concepts,
 ↪  desc="embedding concepts")])
embeddings /= np.linalg.norm(embeddings, axis=1, keepdims=True)

vector_index = faiss.IndexFlatIP(embeddings.shape[1])
vector_index.add(embeddings)
id_by_row = [c["id"] for c in concepts]


def retrieve(query: str, k: int = 5) -> list[tuple[str, float]]:
    q = embed(query)
    q /= np.linalg.norm(q)
    scores, idxs = vector_index.search(q.reshape(1, -1), k)
   return [(id_by_row[i], float(s)) for s, i in zip(scores[0], idxs[0]) if i != -1]
```

```
def llm_rerank(query: str, candidate_ids: list[str], top_n: int = 3) -> list[str]:
    listing = "\n".join(
        f"{i + 1}. {cid} — {concept_by_id[cid]['title']}:
         ↪  {concept_by_id[cid]['description']}"
        for i, cid in enumerate(candidate_ids)
    )
    prompt = f"""A user asked: "{query}"

Candidate knowledge-base entries:
{listing}

List the numbers of the {top_n} most relevant entries, most relevant first, as a
 ↪  comma-separated
list of numbers only (e.g. "2, 5, 1"). Output ONLY the numbers."""
    raw = llm(prompt, num_predict=30, temperature=0.0)
    nums = [int(n) for n in re.findall(r"\d+", raw)]
   ranked = [candidate_ids[n - 1] for n in nums if 0 <= n - 1 < len(candidate_ids)]
    ranked = list(dict.fromkeys(ranked))  # dedupe, preserve order
    ranked += [cid for cid in candidate_ids if cid not in ranked]
    return ranked[:top_n]


def answer_question(query: str, k_retrieve: int = 8, k_final: int = 4,
 ↪  k_raw_guaranteed: int = 2) -> dict:
    """Hybrid selection: the top `k_raw_guaranteed` raw embedding hits always make
     ↪  it into context,
    topped up with the LLM's own reranking. This exists because of a real finding
 ↪  from this
    notebook's first run (see Part 10): a 4B model asked to freely reorder 8
 ↪  candidates sometimes
    drops the single most relevant one — e.g. it ranked `tables/customers` above
 ↪  `tables/orders`
    for a question that was directly about the orders table, even though embedding
 ↪  retrieval had
```

```
    already placed `tables/orders` at rank 2. Trusting dense retrieval as a floor,
 ↪  and using the
    LLM only to fill remaining slots, is a standard defensive pattern for exactly
 ↪  this failure mode.
    """
    hits = retrieve(query, k=k_retrieve)
    candidate_ids = [cid for cid, _ in hits]
    llm_ranked = llm_rerank(query, candidate_ids, top_n=k_final)
    guaranteed = candidate_ids[:k_raw_guaranteed]
    final_context_ids = list(dict.fromkeys(guaranteed + llm_ranked))[:k_final]

    context = "\n\n---\n\n".join(
        f"[{cid}] {concept_by_id[cid]['title']}\n{concept_by_id[cid]['body']}" for
         ↪  cid in final_context_ids
    )
    prompt = f"""Answer the user's question using ONLY the knowledge-base entries
 ↪  below. Cite the
entry id(s) you used in square brackets, e.g. [tables/orders]. If the entries do
 ↪  not contain the
answer, say so explicitly rather than guessing.

Knowledge base entries:
{context}

Question: {query}

Answer (2-4 sentences, with citations):"""
    answer = llm(prompt, num_predict=300, temperature=0.1)
    return {
        "query": query, "retrieved": hits, "llm_ranked": llm_ranked,
        "final_context_ids": final_context_ids, "answer": answer,
    }
```

9. The local RAG discovery agent

```
demo_questions = [
    "What columns does the orders table have and what does one row represent?",
    "How is average order value computed and what is its value?",
    "What percentage of deliveries arrive later than the estimated delivery date?",
    "Which table would I join with order_items to find out who the seller was?",
    "How can I translate product_category_name into English?",
    "What is the distribution of payment types customers use?",
    "How do I find the geographic latitude and longitude for a customer's zip
     ↪  code?",
    "What is the average review score customers give?",
]

demo_results = {}
for q in demo_questions:
    result = answer_question(q)
    demo_results[q] = result
    print("Q:", q)
    print("  retrieved (top-5)  :", [cid for cid, _ in result["retrieved"][:5]])
```

```
    print("  llm_ranked         :", result["llm_ranked"])
    print("  final context used :", result["final_context_ids"])
    print("  answer:", result["answer"])
    print("-" * 110)
```

9.1 Demo — real questions, real answers

```
eval_set = [
    {"question": demo_questions[0], "expected": "tables/orders"},
    {"question": demo_questions[1], "expected":
 ↪  "references/metrics/avg_order_value"},
    {"question": demo_questions[2], "expected":
 ↪  "references/metrics/late_delivery_rate"},
    {"question": demo_questions[3], "expected": "tables/sellers"},
    {"question": demo_questions[4], "expected": "tables/category_translation"},
    {"question": demo_questions[5], "expected":
 ↪  "references/metrics/payment_type_distribution"},
    {"question": demo_questions[6], "expected": "tables/geolocation"},
    {"question": demo_questions[7], "expected":
 ↪  "references/metrics/avg_review_score"},
]


def hit_at_k(expected: str, retrieved_ids: list[str], k: int) -> bool:
    return expected in retrieved_ids[:k]


eval_rows = []
for item in eval_set:
    retrieved_ids = [cid for cid, _ in retrieve(item["question"], k=8)]
    eval_rows.append({
        "question": item["question"][:60] + "...",
        "expected": item["expected"],
        "hit@1": hit_at_k(item["expected"], retrieved_ids, 1),
        "hit@3": hit_at_k(item["expected"], retrieved_ids, 3),
        "hit@5": hit_at_k(item["expected"], retrieved_ids, 5),
        "rank": retrieved_ids.index(item["expected"]) + 1 if item["expected"] in
         ↪  retrieved_ids else None,
    })

eval_df = pd.DataFrame(eval_rows)
display(eval_df)
print(f"\nRetrieval accuracy (embedding only) — Hit@1:
 ↪  {eval_df['hit@1'].mean():.0%}  "
      f"Hit@3: {eval_df['hit@3'].mean():.0%}  Hit@5: {eval_df['hit@5'].mean():.0%}
       ↪  "
      f"(n={len(eval_df)} hand-labeled questions)")
```

10. Evaluation — measured honestly

```
context_hits = []
for item in eval_set:
```

```
    final_ids = demo_results[item["question"]]["final_context_ids"]
    context_hits.append({
        "question": item["question"][:60] + "...",
        "expected": item["expected"],
        "in_final_context": item["expected"] in final_ids,
    })

context_df = pd.DataFrame(context_hits)
display(context_df)
retrieval_hit3 = eval_df["hit@3"].mean()
final_ctx_hit = context_df["in_final_context"].mean()
print(f"\nHit rate — plain retrieval@3: {retrieval_hit3:.0%}   vs.   final context
 ↪  (hybrid, k=4): {final_ctx_hit:.0%}")
if final_ctx_hit < retrieval_hit3:
    print("The LLM rerank step is net-negative on this run: guaranteeing raw top-k
     ↪  hits (Part 9's "
          "`k_raw_guaranteed`) is doing real work, and a larger guaranteed floor
           ↪  may be warranted.")
else:
    print("The hybrid selection preserved (or improved on) plain retrieval
     ↪  accuracy on this run.")
```

```
numeric_checks = [
    (demo_questions[1], metrics_summary["avg_order_value"]),
    (demo_questions[2], metrics_summary["late_delivery_rate"]),
    (demo_questions[7], metrics_summary["avg_review_score"]),
]

print("Answer groundedness (does the generated answer contain the true computed
 ↪  number?):\n")
grounded_flags = []
for q, info in numeric_checks:
    grounded = metric_is_grounded(demo_results[q]["answer"], info)
    grounded_flags.append(grounded)
    print(f"[{'GROUNDED' if grounded else 'NOT GROUNDED'}] {q}")
    print(f"  true value: {format_metric_value(info)} {info['unit']}")
    print(f"  answer    : {demo_results[q]['answer']}\n")

print(f"Groundedness rate: {sum(grounded_flags)}/{len(grounded_flags)}")
```

10.1 Does reranking help or hurt? A direct comparison

11. Mastery recap

Appendix B — Full Source: `agentic_rag_chromadb.ipynb`

Agentic RAG + ChromaDB — the traditional alternative to OKF

```
import json
import os
import re
import time
import warnings
from collections import defaultdict
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Optional

import chromadb
import numpy as np
import ollama
import pandas as pd
from chromadb import EmbeddingFunction
from loguru import logger
from tqdm.auto import tqdm

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 140)

GEN_MODEL = "qwen3.5:4b"
EMBED_MODEL = "nomic-embed-text"
RUN_TS = datetime.now(timezone.utc).replace(microsecond=0).isoformat()

logger.remove()
logger.add(lambda msg: print(msg, end=""), level="INFO",
 ↪  format="<level>{message}</level>\n")


def model_available(name: str, local_models: set) -> bool:
    return name in local_models or any(m.split(":")[0] == name.split(":")[0] for m
     ↪  in local_models)


local_models = {m["model"] for m in ollama.list()["models"]}
for required in (GEN_MODEL, EMBED_MODEL):
    assert model_available(required, local_models), f"Missing local model
     ↪  {required!r}."
print(f"generation model : {GEN_MODEL}\nembedding model  : {EMBED_MODEL}\nrun
 ↪  timestamp    : {RUN_TS}")

CALL_COUNTER = {"llm": 0, "embed": 0}


def llm(prompt: str, *, num_predict: int = 350, temperature: float = 0.2, retries:
 ↪  int = 2) -> str:
    CALL_COUNTER["llm"] += 1
```

```
    last_err = None
    for _ in range(retries + 1):
        try:
            resp = ollama.chat(model=GEN_MODEL, messages=[{"role": "user",
↪   "content": prompt}], think=False,
                               options={"num_predict": num_predict, "temperature":
↪   temperature})
            content = resp["message"]["content"].strip()
            if content:
                return content
        except Exception as e:
            last_err = e
        time.sleep(0.5)
    raise RuntimeError(f"LLM call failed: {last_err}")


def llm_chat(messages: list, *, tools: Optional[list] = None, temperature: float =
↪   0.2):
    CALL_COUNTER["llm"] += 1
    return ollama.chat(model=GEN_MODEL, messages=messages, tools=tools,
     ↪  think=False,
                       options={"temperature": temperature})


class OllamaEmbeddingFunction(EmbeddingFunction):
    """Chroma requires subclassing its `EmbeddingFunction` (not duck-typing) — a
     ↪  bare object with a
    matching `__call__` is missing the `embed_query` mixin method Chroma calls
↪   internally and fails
    with `AttributeError: ... no attribute 'embed_query'` at query time. Verified
↪   directly before
    writing the rest of this notebook."""

    def __init__(self, model: str = EMBED_MODEL):
        self.model = model

    def __call__(self, input):
        CALL_COUNTER["embed"] += len(input)
        return [ollama.embed(model=self.model, input=t)["embeddings"][0] for t in
         ↪  input]

    def name(self):
        return f"ollama-{self.model}"


print(llm("Reply with exactly the word: ready"))
```

0. Environment check

```
import kagglehub

DATASET_DIR = Path(kagglehub.dataset_download("olistbr/brazilian-ecommerce"))
TABLE_RENAME = {"product_category_name_translation": "category_translation"}
```

```
def table_name_from_file(path: Path) -> str:
    name = re.sub(r"_dataset$", "", re.sub(r"^olist_", "", path.stem))
    return TABLE_RENAME.get(name, name)


tables = {table_name_from_file(f): pd.read_csv(f) for f in
 ↪  sorted(DATASET_DIR.glob("*.csv"))}
print(f"Loaded {len(tables)} tables: {list(tables)}")


def profile_table(name: str, df: pd.DataFrame) -> dict:
    cols = []
    for col in df.columns:
        s = df[col]
        cols.append({"name": col, "dtype": str(s.dtype), "null_pct":
 ↪  round(float(s.isna().mean()) * 100, 2),
                     "n_unique": int(s.nunique()), "samples": [str(v) for v in
                      ↪  s.dropna().unique()[:3]]})
    return {"name": name, "n_rows": int(len(df)), "n_cols": int(len(df.columns)),
     ↪  "columns": cols}


profiles = {name: profile_table(name, df) for name, df in tables.items()}


def base_key(col: str) -> str:
    return "zip_code_prefix" if col.endswith("zip_code_prefix") else col


def detect_fk_candidates(tables: dict, min_overlap: float = 0.9) -> pd.DataFrame:
    groups = defaultdict(list)
    for tname, df in tables.items():
        for col in df.columns:
            groups[base_key(col)].append((tname, col))
    rows = []
    for members in groups.values():
        if len(members) < 2:
            continue
        for i, (t1, c1) in enumerate(members):
            for j, (t2, c2) in enumerate(members):
                if i == j:
                    continue
                s1, s2 = tables[t1][c1].dropna(), tables[t2][c2].dropna()
                if s1.empty or s2.empty:
                    continue
                overlap = float(s1.isin(set(s2.unique())).mean())
                if overlap >= min_overlap:
                    rows.append({"from_table": t1, "from_column": c1, "to_table":
 ↪  t2, "to_column": c2,
                                 "overlap": round(overlap, 4)})
    return pd.DataFrame(rows)
```

```
fk_candidates = detect_fk_candidates(tables)
print(f"Detected {len(fk_candidates)} foreign-key candidate relationships.")
```

```
orders_df = tables["orders"].copy()
for col in ["order_purchase_timestamp", "order_delivered_customer_date",
 ↪  "order_estimated_delivery_date"]:
    orders_df[col] = pd.to_datetime(orders_df[col], errors="coerce")

payments_per_order =
 ↪  tables["order_payments"].groupby("order_id")["payment_value"].sum()
delivered = orders_df[orders_df["order_status"] == "delivered"].dropna(
    subset=["order_delivered_customer_date", "order_estimated_delivery_date"])
late_mask = delivered["order_delivered_customer_date"] >
 ↪  delivered["order_estimated_delivery_date"]
review_scores = tables["order_reviews"]["review_score"]
cat_en = tables["products"].merge(tables["category_translation"],
 ↪  on="product_category_name", how="left")
top_categories = (tables["order_items"].merge(cat_en[["product_id",
 ↪  "product_category_name_english"]], on="product_id", how="left")
                  .groupby("product_category_name_english")["order_id"
 ↪  ].nunique().sort_values(ascending=False).head(5))

metrics_summary = {
    "avg_order_value": {"value": round(float(payments_per_order.mean()), 2),
     ↪  "unit": "BRL",
                        "n": int(payments_per_order.shape[0]), "source_tables":
                         ↪  ["order_payments"],
                        "definition": "mean(sum(payment_value) grouped by
                         ↪  order_id)"},
    "late_delivery_rate": {"value": round(float(late_mask.mean()) * 100, 2),
     ↪  "unit": "%",
                           "n": int(len(delivered)), "source_tables": ["orders"],
                           "definition": "mean(delivered_date > estimated_date)
                            ↪  over delivered orders"},
    "avg_review_score": {"value": round(float(review_scores.mean()), 2), "unit":
     ↪  "stars (1-5)",
                         "n": int(review_scores.notna().sum()), "source_tables":
                          ↪  ["order_reviews"],
                         "definition": "mean(review_score)"},
    "review_score_distribution": {"value":
     ↪  (review_scores.value_counts(normalize=True).sort_index() *
     ↪  100).round(2).to_dict(),
                                  "unit": "% of reviews", "n":
                                   ↪  int(review_scores.notna().sum()),
                                 "source_tables": ["order_reviews"], "definition":
                                   ↪  "value_counts(review_score,
                                   ↪  normalize=True)"},
    "payment_type_distribution": {"value":
     ↪  (tables["order_payments"]["payment_type"].value_counts(normalize=True) *
     ↪  100).round(2).to_dict(),
                                  "unit": "% of payments", "n":
                                   ↪  int(tables["order_payments"].shape[0]),
```

```
                                    "source_tables": ["order_payments"],
                                     ↪  "definition": "value_counts(payment_type,
                                     ↪  normalize=True)"},
    "top_product_categories": {"value": top_categories.to_dict(), "unit":
     ↪  "distinct orders", "n": int(top_categories.sum()),
                                    "source_tables": ["order_items", "products",
                                     ↪  "category_translation"],
                                    "definition": "nunique(order_id) grouped by
                                     ↪  product_category_name_english, top 5"},
}
for k, v in metrics_summary.items():
    print(f"{k:28s} = {v['value']}")
```

1. Data, foreign keys, metrics

```
KAGGLE_URL = "https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce"


def format_metric_value(info: dict) -> str:
    v = info["value"]
    return ", ".join(f"{k}: {v2}" for k, v2 in v.items()) if isinstance(v, dict)
     ↪  else str(v)


def number_variants(x: float) -> set:
    return {f"{x}", f"{x:.0f}", f"{x:.1f}", f"{x:.2f}", str(round(x))}


def metric_is_grounded(text: str, info: dict) -> bool:
    clean = text.replace(",", "")
    v = info["value"]
    candidates = v.values() if isinstance(v, dict) else [v]
    return any(variant in clean for x in candidates for variant in
     ↪  number_variants(float(x)))


def make_table_doc(name: str) -> dict:
    profile = profiles[name]
    col_summary = "; ".join(f"{c['name']} ({c['dtype']}, {c['null_pct']}% null)"
 ↪  for c in profile["columns"])
    rels = fk_candidates[(fk_candidates["from_table"] == name) |
 ↪  (fk_candidates["to_table"] == name)]
    rel_bits = [f"{r.from_table}.{r.from_column} relates to
 ↪  {r.to_table}.{r.to_column} ({r.overlap*100:.0f}% overlap)"
                for r in rels.itertuples()]
    prompt = (f"Document a database table. Use ONLY the facts given; never invent
 ↪  numbers or business meaning.\n\n"
              f"Table: {name}\nRows: {profile['n_rows']:,}\nColumns:
               ↪  {col_summary}\n"
              f"Relationships: {'; '.join(rel_bits) if rel_bits else 'none
               ↪  detected'}\n\n"
              f"Write two short paragraphs separated by a blank line: (1) what one
               ↪  row represents, "
```

```
            f"(2) a note on relationships/data quality. Output ONLY the prose.")
    raw = llm(prompt, num_predict=250)
    title = name.replace("_", " ").title()
    text = (f"{title} (table)\n\n{raw}\n\n"
            f"Columns: {col_summary}\n"
            f"Relationships: {'; '.join(rel_bits) if rel_bits else 'none
            ↪ detected'}\n\n"
            f"Source: Olist Brazilian E-Commerce Public Dataset (Kaggle), table
            ↪ `{name}`.")
    return {"id": f"tables/{name}", "text": text, "metadata": {"type": "Table",
    ↪ "title": title}}


def make_metric_doc(key: str) -> dict:
    info = metrics_summary[key]
    value_str = format_metric_value(info)
    prompt = (f"Document a business metric. Metric: {key.replace('_',' ')}\nValue:
↪   {value_str} {info['unit']}\n"
              f"Computed over: {info['n']:,} records\nDefinition:
              ↪ {info['definition']}\n"
              f"Write 2-3 sentences explaining the metric; you MUST include the
              ↪ exact value ({value_str}) verbatim. Output ONLY the prose.")
    text = llm(prompt, num_predict=200)
    if not metric_is_grounded(text, info):
        text = llm(prompt + f"\n\nReminder: {value_str} MUST appear verbatim.",
↪   num_predict=200)
    if not metric_is_grounded(text, info):
        text = f"The {key.replace('_', ' ')} is {value_str}, computed as
↪   {info['definition']} over {info['n']:,} records."
    title = key.replace("_", " ").title()
    full_text = (f"{title} (metric)\n\n{text}\n\n"
                 f"Definition: {info['definition']}\nValue: {value_str}
                 ↪ {info['unit']}\n"
                 f"Computed over: {info['n']:,} records\nSource table(s): {',
                 ↪ '.join(info['source_tables'])}.")
    return {"id": f"references/metrics/{key}", "text": full_text, "metadata":
    ↪ {"type": "Metric", "title": title}}


def make_dataset_doc() -> dict:
    date_min, date_max = orders_df["order_purchase_timestamp"].min(),
↪   orders_df["order_purchase_timestamp"].max()
    valid_years = {date_min.year, date_max.year}
    table_list = "\n".join(f"- {n}: {p['n_rows']:,} rows" for n, p in
↪   profiles.items())
    prompt = (f"Document a dataset. Order date range: {date_min:%Y-%m-%d} to
↪   {date_max:%Y-%m-%d}\nTables:\n{table_list}\n\n"
              f"Write 2-3 sentences on what this dataset represents. Use ONLY the
              ↪ facts above — do not state any "
              f"date range other than the one given. Output ONLY the prose.")
    text = llm(prompt, num_predict=200)

    def years_ok(t):
        return {int(y) for y in re.findall(r"\b(20\d{2})\b",
        ↪ t)}.issubset(valid_years)
```

```
    if not years_ok(text):
        text = llm(prompt + f"\n\nReminder: the only valid years are
↪   {sorted(valid_years)}.", num_predict=200)
    if not years_ok(text):
        text = (f"The Olist Brazilian E-Commerce Public Dataset captures orders
↪   placed between "
                f"{date_min:%B %Y} and {date_max:%B %Y} across {len(profiles)}
                 ↪  tables.")
    full_text = (f"Olist Brazilian E-Commerce
↪   (dataset)\n\n{text}\n\nTables:\n{table_list}\n\n"
                 f"Source: Olist Brazilian E-Commerce Public Dataset (Kaggle),
                  ↪  {KAGGLE_URL}")
    return {"id": "datasets/olist_ecommerce", "text": full_text, "metadata":
     ↪  {"type": "Dataset", "title": "Olist Brazilian E-Commerce"}}
```

```
documents = [make_dataset_doc()]
for name in tqdm(list(profiles), desc="tables"):
    documents.append(make_table_doc(name))
for key in tqdm(list(metrics_summary), desc="metrics"):
    documents.append(make_metric_doc(key))

print(f"\nGenerated {len(documents)} flat documents. LLM calls so far:
↪   {CALL_COUNTER['llm']}")
print("\nExample document (tables/orders):\n")
print(next(d["text"] for d in documents if d["id"] == "tables/orders"))
```

2. Content generation — flat text, not OKF concepts

```
chroma_client = chromadb.EphemeralClient()
collection = chroma_client.get_or_create_collection(
    "olist_flat_corpus", embedding_function=OllamaEmbeddingFunction(EMBED_MODEL)
)
collection.add(
    ids=[d["id"] for d in documents],
    documents=[d["text"] for d in documents],
    metadatas=[d["metadata"] for d in documents],
)
print(f"Ingested {collection.count()} documents into ChromaDB collection
↪   {collection.name!r}.")
```

```
_probe = collection.query(query_texts=["how much do customers spend per order"],
↪   n_results=3)
for i, doc, meta, dist in zip(_probe["ids"][0], _probe["documents"][0],
↪   _probe["metadatas"][0], _probe["distances"][0]):
    print(f"[{i}] {meta['type']} similarity={1 - dist:.3f}")
print("\nChromaDB retrieval sanity check passed." if _probe["ids"][0][0] ==
↪   "references/metrics/avg_order_value"
      else "\nWarning: top hit was not the expected metric doc — inspect above.")
```

3. Ingest into ChromaDB

```
def simple_rag_answer(query: str, top_k: int = 4) -> dict:
    res = collection.query(query_texts=[query], n_results=top_k)
    ids, docs = res["ids"][0], res["documents"][0]
    context = "\n\n---\n\n".join(f"[{i}] {d}" for i, d in zip(ids, docs))
    prompt = (f"Answer the user's question using ONLY the knowledge-base entries
 ↪  below. Cite entry "
              f"id(s) in square brackets, e.g. [tables/orders]. If the entries do
               ↪  not contain the "
              f"answer, say so explicitly rather than guessing.\n\nKnowledge base
               ↪  entries:\n{context}"
              f"\n\nQuestion: {query}\n\nAnswer (2-4 sentences, with citations):")
    answer = llm(prompt, num_predict=300, temperature=0.1)
    return {"retrieved_ids": list(ids), "answer": answer}


print("Simple RAG pipeline ready.")
```

4. Two consumption pipelines

```
SEARCH_TOOL = [{
    "type": "function",
    "function": {
        "name": "search_knowledge_base",
        "description": "Semantic search over the knowledge base. Returns the top
         ↪  matching documents with their id and content.",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "the search query"},
                "top_k": {"type": "integer", "description": "number of results to
                 ↪  return (default 3)"},
            },
            "required": ["query"],
        },
    },
}]

AGENT_SYSTEM_PROMPT = (
    "You are a knowledge-base assistant. Use the search_knowledge_base tool to find
     ↪  relevant "
    "documents before answering. You may call it more than once if the first
     ↪  results are "
    "insufficient — for example, to look up a related table or metric mentioned in
     ↪  an earlier "
    "result. Once you have enough information, answer in 2-4 sentences, citing the
     ↪  document id(s) "
    "you used in square brackets, e.g. [tables/orders]. If nothing relevant turns
     ↪  up after "
    "searching, say so explicitly rather than guessing."
)
```

```
def _run_search_tool(args: dict) -> tuple[str, list]:
    query = args.get("query", "")
    top_k = int(args.get("top_k", 3) or 3)
    res = collection.query(query_texts=[query], n_results=top_k)
    ids, docs = res["ids"][0], res["documents"][0]
    if not ids:
        return "No results found.", []
    return "\n\n".join(f"[{i}]\n{d[:600]}" for i, d in zip(ids, docs)), list(ids)


def agentic_answer(query: str, max_iterations: int = 4) -> dict:
    messages = [{"role": "system", "content": AGENT_SYSTEM_PROMPT}, {"role":
 ↪  "user", "content": query}]
    search_log, retrieved_ids = [], []
    for step in range(max_iterations):
        resp = llm_chat(messages, tools=SEARCH_TOOL)
        messages.append(resp["message"])
        tool_calls = resp["message"].get("tool_calls")
        if not tool_calls:
            return {"answer": resp["message"]["content"], "iterations": step + 1,
             ↪  "searches": search_log,
                    "retrieved_ids": retrieved_ids, "capped": False}
        for tc in tool_calls:
            args = tc.function.arguments
            search_log.append({"query": args.get("query", query), "top_k":
 ↪  args.get("top_k", 3)})
            result, ids = _run_search_tool(args)
            retrieved_ids += [i for i in ids if i not in retrieved_ids]
            messages.append({"role": "tool", "content": result, "tool_name":
 ↪  tc.function.name})

    messages.append({"role": "user", "content": "Answer now with what you already
 ↪  found, citing id(s)."})
    final = llm_chat(messages, tools=None)
    return {"answer": final["message"]["content"], "iterations": max_iterations,
     ↪  "searches": search_log,
            "retrieved_ids": retrieved_ids, "capped": True}


print("Agentic RAG pipeline ready.")
```

4.2 Agentic RAG — the model decides

```
_r = agentic_answer("How much do customers typically spend per basket?")
print("Searches issued :", _r["searches"])
print("Retrieved ids   :", _r["retrieved_ids"])
print("Iterations used :", _r["iterations"], "| hit cap:", _r["capped"])
print("Answer          :", _r["answer"])
```

4.3 Sanity check — watch the agent decide

```
eval_set = [
```

```
    {"q": "What columns does the orders table have?", "expected":
 ↪  {"tables/orders"}, "answerable": True, "category": "direct"},
    {"q": "What is the average order value?", "expected":
 ↪  {"references/metrics/avg_order_value"}, "answerable": True, "category":
 ↪  "direct"},
    {"q": "What is the distribution of payment types?", "expected":
 ↪  {"references/metrics/payment_type_distribution"}, "answerable": True,
 ↪  "category": "direct"},
    {"q": "How can product category names be translated to English?", "expected":
 ↪  {"tables/category_translation"}, "answerable": True, "category": "direct"},
    {"q": "How much do customers typically spend per basket?", "expected":
 ↪  {"references/metrics/avg_order_value"}, "answerable": True, "category":
 ↪  "paraphrase"},
    {"q": "What fraction of shipments arrive behind schedule?", "expected":
 ↪  {"references/metrics/late_delivery_rate"}, "answerable": True, "category":
 ↪  "paraphrase"},
    {"q": "Are Brazilian customers happy with their purchases based on ratings?",
 ↪  "expected": {"references/metrics/avg_review_score",
 ↪  "references/metrics/review_score_distribution"}, "answerable": True,
 ↪  "category": "paraphrase"},
    {"q": "If I know a product's category in Portuguese, how do I find its seller's
 ↪  city?", "expected": {"tables/category_translation", "tables/order_items",
 ↪  "tables/sellers"}, "answerable": True, "category": "multi-hop"},
    {"q": "How would I compute total revenue per seller?", "expected":
 ↪  {"tables/order_items", "tables/sellers", "tables/order_payments"},
 ↪  "answerable": True, "category": "multi-hop"},
    {"q": "Which table tells me both a customer's zip code and their approximate
 ↪  map coordinates?", "expected": {"tables/customers", "tables/geolocation"},
 ↪  "answerable": True, "category": "multi-hop"},
    {"q": "seller performance", "expected": {"tables/sellers",
 ↪  "tables/order_items"}, "answerable": True, "category": "vague"},
    {"q": "payment methods", "expected":
 ↪  {"references/metrics/payment_type_distribution", "tables/order_payments"},
 ↪  "answerable": True, "category": "vague"},
    {"q": "What is the customer's email address or phone number?", "expected":
 ↪  set(), "answerable": False, "category": "distractor"},
    {"q": "What was Olist's total company revenue in 2020?", "expected": set(),
 ↪  "answerable": False, "category": "distractor"},
]
pd.DataFrame([{"category": e["category"], "answerable": e["answerable"],
 ↪  "question": e["q"]} for e in eval_set])
```

5. Evaluation set

```
REFUSAL_RE = re.compile(
    r"do(?:es)? not (?:contain|include|provide)|not available|cannot determine|no
     ↪  information|"
    r"don'?t have|isn'?t available|unable to (?:determine|answer)|not
     ↪  (?:mentioned|specified|available)|"
    r"knowledge base (?:entries )?do(?:es)? not|no results found",
    re.IGNORECASE,
)
```

```
def looks_like_refusal(answer: str) -> bool:
    return bool(REFUSAL_RE.search(answer))


def hit_at_k(expected: set, ids: list, k: int):
    return any(cid in expected for cid in ids[:k]) if expected else None


def cite_hit(expected: set, answer: str, known_ids: list):
    if not expected:
        return None
    return any(f"[{cid}]" in answer for cid in known_ids if cid in expected)


bench_rows = []
for item in tqdm(eval_set, desc="benchmark"):
    q, expected, answerable, category = item["q"], item["expected"],
 ↪  item["answerable"], item["category"]

    before = dict(CALL_COUNTER)
    s = simple_rag_answer(q)
    simple_calls = (CALL_COUNTER["llm"] - before["llm"]) + (CALL_COUNTER["embed"]
 ↪  - before["embed"])

    before = dict(CALL_COUNTER)
    a = agentic_answer(q)
    agentic_calls = (CALL_COUNTER["llm"] - before["llm"]) + (CALL_COUNTER["embed"]
 ↪  - before["embed"])

    all_ids = [d["id"] for d in documents]
    bench_rows.append({
        "question": q[:55], "category": category, "answerable": answerable,
        "simple_hit@1": hit_at_k(expected, s["retrieved_ids"], 1),
        "simple_hit@3": hit_at_k(expected, s["retrieved_ids"], 3),
        "simple_cite_hit": cite_hit(expected, s["answer"], all_ids),
        "agentic_search_hit": hit_at_k(expected, a["retrieved_ids"],
         ↪  len(a["retrieved_ids"])),
        "agentic_cite_hit": cite_hit(expected, a["answer"], all_ids),
        "agentic_n_searches": len(a["searches"]), "agentic_capped": a["capped"],
        "simple_refusal_ok": (not answerable) and looks_like_refusal(s["answer"]),
       "agentic_refusal_ok": (not answerable) and looks_like_refusal(a["answer"]),
        "simple_calls": simple_calls, "agentic_calls": agentic_calls,
    })

bench_df = pd.DataFrame(bench_rows)
display(bench_df)
```

6. Head-to-head benchmark

```
answerable_df = bench_df[bench_df["answerable"]]
unanswerable_df = bench_df[~bench_df["answerable"]]
```

```
summary = pd.DataFrame({
    "simple RAG": [
       answerable_df["simple_hit@1"].mean(), answerable_df["simple_hit@3"].mean(),
        answerable_df["simple_cite_hit"].mean(), float("nan"),
        unanswerable_df["simple_refusal_ok"].mean(),
    bench_df["simple_calls"].mean(),
    ],
    "agentic RAG": [
        answerable_df["agentic_search_hit"].mean(), float("nan"),
        answerable_df["agentic_cite_hit"].mean(),
    answerable_df["agentic_n_searches"].mean(),
        unanswerable_df["agentic_refusal_ok"].mean(),
    bench_df["agentic_calls"].mean(),
    ],
}, index=["Retrieval hit rate", "Hit@3 (simple only)", "Answer-citation hit rate",
          "Avg searches issued (agentic only)", "Refusal correctness
           (distractors)", "Avg LLM+embed calls / question"])
display(summary.round(3))

print(f"\nn={len(answerable_df)} answerable questions, n={len(unanswerable_df)}
    distractor questions.")
print(f"Agentic pipeline issued {answerable_df['agentic_n_searches'].mean():.1f}
    searches/question on average "
      f"(cap=4); {int(bench_df['agentic_capped'].sum())}/{len(bench_df)} questions
       hit the iteration cap.")
print(f"Agentic pipeline cost {summary.loc['Avg LLM+embed calls / question',
    'agentic RAG'] / summary.loc['Avg LLM+embed calls / question', 'simple
    RAG']:.1f}x "
      f"the calls of simple RAG per question — measured, not estimated.")
```

6.1 Aggregate scorecard

```
by_category = answerable_df.groupby("category")[["simple_cite_hit",
    "agentic_cite_hit", "agentic_n_searches"]].mean()
display(by_category.round(2))
```

6.2 Breakdown by question category

```
def delta_word(base: float, adv: float) -> str:
    if adv == base:
        return "tied with"
    return "beat" if adv > base else "underperformed"


s_cite, a_cite = answerable_df["simple_cite_hit"].mean(),
    answerable_df["agentic_cite_hit"].mean()
s_ref, a_ref = unanswerable_df["simple_refusal_ok"].mean(),
    unanswerable_df["agentic_refusal_ok"].mean()
s_cost, a_cost = bench_df["simple_calls"].mean(), bench_df["agentic_calls"].mean()

print(f"Answer-citation hit rate : agentic {delta_word(s_cite, a_cite)} simple —
    {a_cite:.0%} vs {s_cite:.0%}")
```

```
print(f"Distractor refusal       : agentic {delta_word(s_ref, a_ref)} simple —
 ↪  {a_ref:.0%} vs {s_ref:.0%}")
print(f"Cost                     : agentic used {a_cost / s_cost:.1f}x the calls of
 ↪  simple RAG")
print(f"Average searches issued by the agent per question:
 ↪  {answerable_df['agentic_n_searches'].mean():.1f} "
      f"(1 search = behaviorally identical to simple RAG; >1 means it decided to
       ↪  look further)")
```

7. Honest verdict

7.1 What actually happened when the model could choose to search again

8. OKF vs. flat-corpus RAG+VectorDB — lessons from actually building both

Appendix C — Environment & Setup Reference

```
# Environment
uv sync
ollama pull qwen3.5:4b
ollama pull nomic-embed-text

# Kaggle credentials (either form)
#   ~/.kaggle/kaggle.json
#   or: export KAGGLE_USERNAME=... KAGGLE_KEY=...

# Run
uv run jupyter lab google_okf_zero_to_mastery.ipynb
uv run jupyter lab agentic_rag_chromadb.ipynb
```

Models used throughout, both ≤4B parameters, both local via Ollama:

Role	Model	Notes
Generation	`qwen3.5:4b`	Hybrid-reasoning model — `think=False` is required, or it can spend its entire token budget on hidden `<think>` traces and return empty content (a real issue hit during development).
Embeddings	`nomic-embed-text`	768-dimensional embeddings, used identically by both notebooks.

Key dependencies: `pandas`, `pyyaml`, `ollama`, `kagglehub`, `faiss-cpu`, `pyvis`, `rank-bm25`, `chromadb`, `numpy`, `tqdm`, `loguru` — see `pyproject.toml`/`uv.lock` for exact pinned versions.

License: the code in this repository is MIT-licensed (see `LICENSE`). The Olist dataset is CC BY-NC-SA 4.0 and is downloaded at runtime, never redistributed in this repository.